Switchboard — Product Requirements Document

Ambient shared working memory for concurrent AI coding sessions in one workspace.

Field	Value
Codename	**Switchboard** (name TBD — see §20 Open Questions)
Install	`npx switchboard init`
Version	Draft v1.0
Date	2026-08-25
Author	Aayush
Status	Pre-build — for review and scoping
Target platform	**Agent-agnostic** — any MCP-capable coding agent. Enhanced on hook-capable agents (Claude Code, Cursor, Gemini CLI, OpenCode). macOS, Linux, WSL2

---

0. TL;DR

You open five Claude Code sessions in one VS Code workspace. Each one starts blind. None of them knows the other four exist, what they've discovered, or which files they're mid-edit on. **You become the message bus** — copy-pasting findings, re-explaining the same architecture five times, and refereeing conflicting edits after the fact.

Switchboard is a zero-config package that turns those independently-opened sessions into a shared-awareness group. It does three things:

1. **Cold-start priming** — a new session opens already knowing what the other live sessions are working on.
2. **Directed handoff** — pull another live session's working context on demand, without copy-paste.
3. **Collision guard** — a session is warned before editing a file another session has claimed.

The architecture is deliberately small, and deliberately agent-agnostic: one MCP server + one SQLite file, with lifecycle hooks as an optional enhancement. No daemon. No VS Code extension. No new transport protocol. **No per-agent adapters.**

The key structural insight: **an MCP server is itself a universal session registry.** Every MCP-capable agent spawns the server per session and announces itself in the `initialize` handshake — so discovery, liveness, and identity come free on every agent that speaks MCP, which by mid-2026 is effectively all of them. Hooks are not the foundation; they are a capability upgrade on the two agents that have them.

The critical strategic fact: on 2026-08-08, Anthropic shipped native cross-session messaging. It solves the *transport*. It explicitly does **not** solve the *state* — it sends plain text between two sessions on demand, carries no history, persists nothing, and never primes a new session at startup. That is precisely the gap

this product fills, and it now defines a much sharper, much smaller product than it would have three weeks ago.

---

1. Problem

1.1 The scenario

One repo. One VS Code window. Five Claude Code sessions, opened over three hours as work branched:

- Session A: debugging an auth failure. Has read 14 files, run 30 greps, and finally found the bug at `middleware/session.ts:142`.
- Session B: writing tests for the payments module.
- Session C: you just opened it, to refactor the API client.

Session C knows nothing. It doesn't know A found the auth bug. It doesn't know B is mid-edit on three files C is about to touch. It doesn't know the team decided against Redis last Tuesday, in session A's context window, and nowhere else.

So you paste. You re-explain. And when C overwrites B's edit, you debug a merge conflict that a single sentence would have prevented.

1.2 The four costs

Cost	Mechanism	Who pays
Rediscovery tokens	Every fresh session re-reads the same files and re-runs the same searches to reach the same understanding	Your API bill
Human relay	You are the router between sessions — reading A's output, summarizing it, pasting into C	Your attention, continuously
Write collisions	Two sessions edit the same file with no mutual awareness	Correctness; debugging time
Compaction amnesia	Session A compacts. Hard-won context degrades. There is no external store to recover it from	Everyone, silently

1.3 The precise framing

The human is currently the message bus. Not metaphorically — literally. You perform routing, summarization, and conflict detection by hand, with your eyes, across terminal tabs.

Every other cost in this document is downstream of that one sentence. The product's entire job is to make that role unnecessary.

1.4 Evidence this is real

From this machine, right now:

```
~/.claude/projects/-Users-swapnanilm-Desktop-logline/
  → 20 session transcripts, one workspace
  → 8 of them 3–14 MB each
  → all last-written 2026-08-25 17:47 — the same minute
```

Twenty sessions in a single project directory, several written to simultaneously. This is not a hypothetical workflow. It is the observed default for anyone doing serious agent-assisted work, and every one of those sessions started cold.

The upstream feature request (anthropics/claude-code#21277) names the same four pain points independently — duplicated exploration, conflicting changes, lost context on switching, and inefficient handoffs — and was closed as a duplicate, meaning it has been filed more than once.

---

2. Why now

1. **Multi-session is the default workflow, not a power-user edge case.** Agent work naturally branches. People open tabs faster than they close them.
2. **The hook API became sufficient.** `SessionStart` gained silent context injection via `hookSpecificOutput.additionalContext` (Claude Code 2.1.0+). Before that, priming a session meant an extension or a wrapper. Now it's a config entry.
3. **Claude Code already maintains a live session registry.** `~/.claude/sessions/<pid>.json` records `cwd`, `status`, `name`, `sessionId`, and heartbeats. Discovery is a directory read. This did not exist a year ago.
4. **Anthropic validated the problem and left the hard half open.** Shipping cross-session messaging proves the pain is real and worth platform investment. Its documented limits define exactly where a third-party layer still has room.
5. **Context windows got large enough that the bottleneck moved.** The constraint is no longer "can this fit" — it is "does this session know it should look."

---

3. Prior art and competitive landscape

3.1 Native Claude Code features (the real competition)

Cross-session messaging — shipped 2026-08-08, requires v2.1.224+, on by default where available. Claude uses `ListAgents` to discover peers and `SendMessage` to deliver text. Delivery is over a per-session Unix socket, never through Anthropic servers on the same machine.

What the documentation states it does **not** do — each line is a product opening:

> "A message is a piece of text one Claude writes to another, **never conversation history or files**."

> "the receiving session gets only that text, **never the sender's conversation history**"

- **On-demand only.** Someone — you or the model — must decide to send. Nothing is ambient.
- **No persistence.** A message is delivered and gone. Nothing survives the session.
- **No startup priming.** A brand-new session learns nothing automatically. This is the user's single loudest pain point, and native messaging does not touch it.
- **Plain text only.** No structured state, no file claims, no queryable history.
- **Point-to-point.** No shared surface; N sessions means N−1 sends, one per recipient.

> ⚠️ **Note for this machine:** you are on **v2.1.223**. Cross-session messaging requires **v2.1.224**. You do not have it yet — you are one patch version away. Upgrade before validating any of the assumptions in this document.

Agent Teams — experimental, requires `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`. A lead session spawns teammates who share a task list with file locking and dependency tracking. Genuinely good at what it does. It is a **different shape of problem**:

> "**One team per session**: a session has exactly one team, scoped to that session. You can't create additional named teams or **share a team across sessions**."

> "**Lead is fixed**: the main session is the lead for its lifetime."

This is the decisive distinction. **Agent Teams is a planned org chart, spawned top-down from a lead. Independently-opened sessions cannot join one retroactively.** Your five hand-opened tabs are structurally ineligible. Agent Teams also requires you to know the shape of the work upfront — which is exactly what you don't know when work branches organically.

Additional friction: it is off by default, experimental, does not survive `/resume` for in-process teammates, and once enabled, silently converts named subagents into teammates — which changes unrelated delegation behavior.

Session resume (`--resume`) — restores *one* conversation. Serial continuation, not concurrent sharing.

3.2 Third-party memory layers

Tool	Model	Gap for this use case
mem0	Hosted MCP memory, semantic recall	Long-term facts across time. Not live cross-session state. Cloud-hosted.
claude-mem	Local hooks + compression	Same-session continuity over time, not peer awareness. A Feb 2026 community audit rated it **HIGH security risk** — unauthenticated HTTP API on port 37777, allowing any local process to read stored observations and inject memories. **A cautionary precedent this design must not repeat** (see §16).
Cipher / ByteRover	Cloud-stored, team-shareable	Team knowledge base. Cloud dependency. Not workspace-live-session-scoped.
CLAUDE.md / AGENTS.md	Static file, loaded at start	Correct for durable facts. Wrong medium for state that changes every 90 seconds.
session-bridge	9 bash scripts, file inbox/outbox, manual 6-char ID copy-paste	Closest hobbyist precursor. Proves the demand. Manual pairing, no persistence, 5–10s latency, now largely superseded by native messaging.

Synthesis: every existing tool addresses either *long-term project memory* (weeks) or *point-to-point messaging* (instant, ephemeral). Nothing occupies the middle.

3.3 The memory tier map — where the gap is

Tier	Lifetime	Owned by	Example
T0 · Repo knowledge	Forever	CLAUDE.md, AGENTS.md	"we use pnpm, tests are vitest"
T1 · Project memory	Weeks	mem0, Cipher, claude-mem	"we decided against Redis in March"
T2 · Live working set	**Hours — this work session**	⛔ **nothing**	**"A is mid-refactor on `auth/*`, found the bug at `session.ts:142`, don't touch it"**
T3 · Turn context	Minutes	the context window	the current conversation

T2 is the gap, and it is unclaimed. It's awkward territory: too ephemeral to justify a vector database, too cross-cutting to live in any one context window, too dynamic for a markdown file. That awkwardness is why it's still open.

Native messaging is a *pipe* laid across T2. It does not create the *shared surface* T2 needs.

Sources: cross-session messaging · agent teams · hooks reference · issue #21277 · the-decoder launch coverage · session-bridge · claude-mem · mem0

---

4. The wedge

> Native messaging gives your sessions a **telephone**. Switchboard gives them a **shared whiteboard in the room** — one that was already covered in writing before the newest person walked in.

Capability	Native messaging	Agent Teams	Switchboard
Adopts sessions **you** opened, unplanned	✅	❌	✅
Auto-primes a new session at startup	❌	⚠️ spawn prompt only	✅
Ambient — nobody must decide to send	❌	⚠️	✅
Survives a session's death	❌	⚠️ task list only	✅
Structured, queryable state	❌ text only	⚠️ tasks only	✅
File-claim collision guard, ad-hoc sessions	❌	✅ within team	✅
Enabled by default, zero config	✅	❌	✅ after `init`
Works with no team lead	✅	❌	✅

One-sentence positioning: *Switchboard is the ambient state layer that native cross-session messaging assumes you already have.*

4.1 The honest scope reduction

Three weeks ago this product would have needed its own transport, its own discovery, and its own delivery guarantees. Anthropic shipped all three on August 8.

The correct response is to delete that half of the product and use theirs. Switchboard does not build a message bus. It builds the shared state, and hands off delivery to `SendMessage` and the existing peer socket.

This is not a concession. It cuts the build roughly in half, removes the entire class of transport bugs, and means Switchboard gets better every time Anthropic improves messaging instead of competing with it.

---

5. Product principles

1. **Zero config or it doesn't ship.** One `npx` command. No accounts, no daemon to babysit, no env var to remember. Adoption dies at step two.
2. **Ambient by default, explicit when it matters.** Passive awareness is automatic. Pulling a full context dump is deliberate.
3. **Context is a budget, not a bucket.** Every injected token displaces a token of real work. Injection is capped, measured, and defensible. **Polluting context is worse than injecting nothing.**
4. **Observe, don't ask.** File claims come from watching `Edit` / `Write` tool calls, not from asking the agent to self-report. Agents forget; hooks don't.

5. **Never leave the machine.** Transcripts contain secrets, customer data, and unreleased code. Local-only, no exceptions, no telemetry on content.
6. **Ride the platform, never fight it.** Every capability that Claude Code ships natively — discovery, sockets, injection, liveness — is used, not reimplemented. When the platform absorbs a Switchboard feature, delete it and celebrate.
7. **Degrade to invisible.** If the store is corrupt, the version is unsupported, or a hook times out — exit 0, inject nothing. A context-sharing tool must never be the reason a session fails to start.
8. **Earn every token.** If a user cannot point at a moment Switchboard saved them, it is uninstall-bait.

---

6. Users and jobs-to-be-done

Primary — the multi-tab solo developer. Runs 3–8 concurrent sessions in one workspace. Currently the message bus. Wants to stop being it.

Secondary — the parallel-worktree developer. Sessions across git worktrees on the same repo. Needs to know what landed where.

Tertiary — the small team on a shared machine or devbox. Multiple humans, shared filesystem. Out of scope for V1; informs the data model so V2 isn't a rewrite.

Jobs

#	Job	Priority
JTBD-1	When I open a new session, it should already know what's happening — so I don't re-explain	**P0**
JTBD-2	When I'm about to edit a file another session owns, warn me *before* the edit	**P0**
JTBD-3	When one session solves something, others should be able to find that answer without me relaying it	**P0**
JTBD-4	Give this new session everything that session knows, right now, on my command	P1
JTBD-5	Show me what all my sessions are doing, in one view	P1
JTBD-6	When a session dies or compacts, don't lose what it learned	P1
JTBD-7	Let a session ask "has anyone already tried this?" before spending 20k tokens	P2

---

7. Scope

V0 — Prove the wedge (target: 2 weeks)

Ships JTBD-1, 2, 3. This is the whole bet; everything after is amplification.

- `switchboard-mcp` **server** — the universal substrate: self-registration on `initialize`, liveness by process lifetime, workspace scoping by `cwd`
- **MCP pull tools** — `sb_peers`, `sb_session`, `sb_search`, `sb_claim`
- `npx switchboard init` — detects every installed agent, writes MCP config to each, writes hooks where supported, appends the AGENTS.md stanza, creates the store. Idempotent, prints a diff
- `.switchboard/ROSTER.md` — live roster as a plain file, readable by any agent, git-excluded
- **Session roster injection** at session start — hard-capped digest (T3 automatic; T0/T2 on all others)
- **File-claim tracking** — observed on T3, voluntary via `sb_claim` on T1
- **Collision warning** — non-blocking, warn only, never deny
- **Session digest** on stop/end — mechanical extraction, no LLM call
- `npx switchboard status` — agent-labelled roster
- `npx switchboard doctor` — per-agent tier report
- `npx switchboard uninstall` — reverses `init` across every agent it touched

V1 — Make it good (weeks 3–6)

- `sb_handoff` — directed context transfer (JTBD-4), delivered over native `SendMessage`
- `sb_note` — a session posts a durable finding to the commons ("auth bug is at `session.ts:142`")
- `PreCompact` **rescue** — persist the working set before compaction destroys it (JTBD-6)
- **Delta injection** at `UserPromptSubmit` — only what changed since this session's last turn, strictly rate-limited
- `npx switchboard board` — live TUI roster (JTBD-5)
- **Effectiveness telemetry** — local-only; injected tokens vs. tool calls avoided

V2 — Expand (weeks 7+)

- Cross-worktree awareness via git root instead of `cwd`
- Multi-human shared-machine mode, with attribution
- Optional LLM-summarized digests via Haiku when mechanical extraction proves too thin

Explicit non-goals

Not building	Why
A VS Code extension	The MCP server attaches to the *agent*, not the editor. An extension buys nothing and costs marketplace review, a UI surface, and editor lock-in.
Per-agent adapters or plugins	MCP + AGENTS.md are already universal. Writing N adapters means maintaining N adapters forever, and still missing agent N+1.
A background daemon	Nothing to supervise, crash, orphan, or leak. Hooks are the scheduler.
A new message transport	Native `SendMessage` and the peer socket exist and are better than what we'd build.
Cloud sync or an account	Kills the "never leaves the machine" guarantee, which is the trust anchor.
A vector DB / semantic search	T2 data is small and recent. SQLite FTS5 is enough. Revisit only if measurably insufficient.
Full transcript sharing	100k-token dumps make agents *worse*. Distillation is the product.
Task orchestration / a lead agent	That's Agent Teams. Don't compete with the platform on its own turf.
Auto-resolving edit conflicts	Warn, never merge. Silent merges destroy trust permanently.
Windows native (V0)	Named pipes and a different home dir. Defer; WSL2 works.

---

8. Functional requirements

Install

ID	Requirement	Priority
FR-1.1	`npx switchboard init` completes in <10s with no prompts	P0
FR-1.2	Merges into existing `.claude/settings.json` without clobbering user hooks	P0
FR-1.3	Idempotent — re-running changes nothing	P0
FR-1.4	Prints a diff of what it changed; `--dry-run` supported	P0
FR-1.5	Detects Claude Code version; warns and no-ops on unsupported	P0
FR-1.6	`uninstall` removes all hooks, MCP entries, and optionally the store	P0
FR-1.7	Adds its store dir to `.git/info/exclude` — never touches the user's `.gitignore`	P1

Discovery and liveness

ID	Requirement	Priority
FR-2.1	Discover peers by reading `~/.claude/sessions/*.json`, filtered to matching `cwd`	P0
FR-2.2	Liveness = PID alive **AND** heartbeat within TTL (default 120s)	P0
FR-2.3	Group by VS Code workspace via `~/.claude/ide/*.lock` `workspaceFolders` when present, else `cwd`	P0
FR-2.4	Reap stale sessions and release their file claims automatically	P0
FR-2.5	Never treat a dead session's claim as blocking	P0

Cold-start priming — *the core feature*

ID	Requirement	Priority
FR-3.1	On `SessionStart`, inject a digest of live peers in this workspace	P0
FR-3.2	Digest is **hard-capped at 500 tokens**, truncated oldest-first	P0
FR-3.3	Digest contains, per peer: name, status, current focus, claimed files, last finding	P0
FR-3.4	Inject **nothing** when no live peers exist — zero cost in the single-session case	P0
FR-3.5	Digest states explicitly that richer detail is available via `sb_*` tools	P0
FR-3.6	Hook completes in <500ms or yields, injecting nothing	P0
FR-3.7	On `matcher: resume` / `compact`, inject only the delta since last seen	P1

Collision guard

ID	Requirement	Priority
FR-4.1	`PostToolUse` on `Edit|Write|NotebookEdit` records a file claim with a TTL	P0
FR-4.2	`PreToolUse` on the same tools checks for a live foreign claim	P0
FR-4.3	On conflict, inject a warning via `additionalContext`. `permissionDecision` **is never** `deny`	P0
FR-4.4	Warning names the peer, the file, and the age of the claim	P0
FR-4.5	Claims expire (default 15 min) and release on `SessionEnd`	P0
FR-4.6	Same-session re-edits never warn	P0
FR-4.7	`--strict` opt-in mode may deny. **Off by default.**	P2

Pull interface (MCP)

ID	Tool	Returns	Priority
FR-5.1	`sb_peers()`	Live sessions, focus, claims	P0
FR-5.2	`sb_session(name)`	One peer's full digest: files, decisions, findings	P0
FR-5.3	`sb_search(query)`	FTS5 across digests and notes — "has anyone tried X?"	P0
FR-5.4	`sb_note(text, tags)`	Post a durable finding to the commons	P1
FR-5.5	`sb_handoff(target, scope)`	Push this session's context to a named peer via `SendMessage`	P1
FR-5.6	`sb_claim(paths, reason)`	Explicitly reserve files before a large refactor	P2

Digest generation

ID	Requirement	Priority
FR-6.1	Digests are generated **mechanically** from hook payloads — no LLM call in V0	P0
FR-6.2	Extract: files read/written, commands run, `last_assistant_message` from `Stop`, first user prompt as intent	P0
FR-6.3	Regenerate on every `Stop`; cap per-session digest at 2k tokens	P0
FR-6.4	`PreCompact` snapshots the working set before compaction	P1
FR-6.5	Optional Haiku summarization, opt-in, never blocking	P2

---

9. UX and interaction design

9.1 What the user actually sees

Almost nothing. That's the design.

The primary interface is a Claude session that already knows things. Success is the *absence* of an experience — no re-explaining, no copy-paste, no post-hoc merge conflict.

9.2 The injected digest

Format matters enormously — this text competes with the user's actual work for context. Target ~180 tokens for a 3-peer workspace:

```
[switchboard] 3 live sessions in this workspace:

• auth-fix (active, 40m) — debugging login 500s
  editing: middleware/session.ts, lib/jwt.ts
  latest: root cause is the exp claim compared in ms not seconds

• payments-tests (idle 6m) — writing tests for checkout
  editing: __tests__/checkout.spec.ts

• api-refactor (active, 3m) — no claims yet

Use sb_session(name) for detail, sb_search(q) to check prior work.
Files above are claimed by other sessions — check before editing.
```

Design rules, each earning its place:

- **Named, not UUID'd.** Uses Claude Code's own session names (`logline2-69`) — they're already in the registry and already what `@`-mention uses.
- **Recency-ordered**, oldest truncated first.
- **A pointer, not a payload.** The last two lines are the highest-value tokens in the block: they tell the model where to get more instead of pre-loading it.
- **Prefixed** `[switchboard]` so it's greppable in transcripts and obvious in origin.

9.3 Collision warning

Fires at `PreToolUse`, before the edit lands:

```
[switchboard] ⚠ middleware/session.ts is claimed by session "auth-fix"
(active, last edited 4m ago). They're debugging login 500s.
Consider sb_session("auth-fix") first, or coordinate before editing.
```

Warn, never block. A false-positive block is infuriating and gets the tool uninstalled within a day. A false-positive warning costs 40 tokens.

9.4 CLI surface

```
npx switchboard init          # install hooks + MCP into this workspace
npx switchboard status        # roster, one screen
npx switchboard board         # live TUI (V1)
npx switchboard doctor        # diagnose: version, hooks, store, permissions
npx switchboard uninstall     # full reversal
```

`doctor` is not optional polish. Hook-based tools fail silently by design, which makes them undebuggable without one.

---

10. Technical architecture

10.1 The shape



There is still no daemon. The MCP server process is spawned and reaped by the agent itself — its lifetime *is* the session's lifetime, which is precisely the liveness signal we need. Nothing is supervised, nothing is orphaned, nothing listens on a port.

10.2 Why the MCP server is the universal substrate

This is the decision that makes cross-agent support native rather than adapter-based.

Under stdio transport, an MCP client spawns **one server process per session** and terminates it when the session ends. The `initialize` handshake carries `clientInfo: { name, version }`. Therefore, for free, on every MCP-capable agent:

Need	Comes from	Per-agent code required
Session discovery	server process starts	none
Session identity	`initialize` → `clientInfo.name`	none
Workspace scoping	server's `cwd`	none
Liveness	process alive	none
Death detection	process exits	none
Pull interface	MCP tools	none
Write interface	MCP tools	none

Zero adapters. Not "a thin shim per agent" — genuinely zero. An agent that speaks MCP works on the day it ships, without Switchboard knowing it exists.

This inverts the original design. Hooks were going to be the foundation with MCP as a convenience; in fact MCP is the foundation and hooks are the convenience.

10.3 The four-tier capability model

Every tier degrades cleanly to the one below it. No agent is excluded; agents differ only in how much is automatic.

Tier	Mechanism	Works on	Gives you
T0 · Instruction	one static stanza appended to `AGENTS.md` at `init`	**30+ agents** that read AGENTS.md natively	the agent knows the commons exists and how to reach it
T1 · Pull	MCP tools	**every MCP client** (~14+ agents)	full read/write on shared state, on demand
T2 · Passive read	`.switchboard/ROSTER.md`, rewritten on change	**literally any agent** — it's a file	cold-start roster even with no MCP and no hooks
T3 · Automatic	lifecycle hooks	**Claude Code, Cursor, Gemini CLI, OpenCode**	silent auto-injection at startup, observed file claims, collision guard

T0 is written once and never churns. `init` appends a short committed stanza to `AGENTS.md`:

```
## Concurrent sessions
Other AI sessions may be working in this repo right now.
Before starting a task, call `sb_peers` (MCP) — or read
`.switchboard/ROSTER.md` — to see what they're doing and which
files they've claimed. Post findings with `sb_note`.
```

That is the entire cross-agent integration. It is a static instruction in a file that 30+ agents already load at session start. The churning state lives in `.switchboard/ROSTER.md`, which is git-excluded, so the repo sees no diff noise.

> **Claude Code caveat:** as of August 2026 Claude Code loads `CLAUDE.md`, not `AGENTS.md`. `init` detects this and writes `@AGENTS.md` as the first line of `CLAUDE.md` — the bridge Anthropic's own docs recommend — or the stanza directly if `CLAUDE.md` already exists with content.

10.4 Honest capability differences

Selling "identical everywhere" would be a lie that surfaces during implementation. One capability genuinely differs:

File-claim attribution requires either hooks or voluntary calls.

- **T3 agents (Claude Code, Cursor, Gemini CLI, OpenCode):** claims are *observed*. The post-tool hook fires on every successful edit whether or not the model remembered to announce it. Reliable.
- **T1 agents (everyone else):** claims are *voluntary* — the model calls `sb_claim()` because the T0 stanza told it to. Good, not guaranteed.

There is no universal fix. Filesystem watching detects *that* a file changed but cannot attribute it to a session; git status has the same flaw. **Attribution requires interception, and only hooks intercept.** This is documented as a tier difference, not papered over.

Everything else — discovery, roster, digests, search, notes, handoff — is identical on every tier.

10.5 Why this is the lazy architecture

Each rung climbed, and what it saved:

Naive design	What actually exists	Saved
A shim per agent for discovery	MCP `initialize` + process lifetime — universal	N adapters, and all future ones
VS Code extension for session discovery	`~/.claude/sessions/*.json`; MCP server `cwd` elsewhere	Entire extension + marketplace review
WebSocket daemon + registry	Agent-spawned MCP process + SQLite WAL	Daemon lifecycle, ports, orphans, supervision
Custom injection mechanism	`SessionStart.additionalContext` (T3); `AGENTS.md` (T0)	The hardest part of the product, free
A cross-agent context-file format	**AGENTS.md** — already a Linux Foundation standard read by 30+ agents	Inventing a standard nobody adopts
Custom message transport	Native `SendMessage` + peer socket	~40% of the build, and all transport bugs
Redis / Postgres	SQLite WAL, `node:sqlite` (Node 22+), zero deps	An install step users would refuse
Vector DB for search	SQLite FTS5, built in	A dependency, an index, an embedding cost
Agent self-reports file edits	`PostToolUse` / `afterFileEdit` observes them (T3)	Reliability — agents forget, hooks don't

Net: ~1,800 lines, zero runtime dependencies, one file on disk, every agent supported.

This is the single most important section of the document. Nearly every failed version of this idea fails by building the daemon, or by writing one adapter per agent and drowning in maintenance.

10.6 Evidence the primitives exist

Verified on this machine, 2026-08-25:

```
// ~/.claude/sessions/19658.json — the discovery substrate
{
  "pid": 19658,
  "sessionId": "34d253f1-5692-462f-91bc-fbdb08a0c0b6",
  "cwd": "/Users/swapnanilm/Desktop/logline2",   // ← workspace grouping
  "startedAt": 1787644923700,
  "version": "2.1.223",
  "kind": "interactive",
  "name": "logline2-69",                         // ← human-addressable
  "status": "waiting",                           // ← live status
  "updatedAt": 1787660510478,                    // ← heartbeat, liveness
  "waitingFor": "permission prompt"
}
```

```
// ~/.claude/ide/12864.lock — the VS Code workspace binding
{
  "pid": 3197,
  "workspaceFolders": ["/Users/swapnanilm/Desktop/logline2"],
  "ideName": "Visual Studio Code"
}
```

Discovery, liveness, naming, status, and workspace grouping are **all already on disk**. Switchboard reads them. It writes nothing to `~/.claude/` outside its own config entries.

> ⚠️ These paths are **undocumented internals**. See RISK-1 in §17 for the mitigation — this is the product's most serious technical dependency.

10.7 Hook wiring (T3 enhancement)

Four agents expose hook lifecycles, and they converged on near-identical shapes — plainly modelled on each other. One implementation serves all four behind a ~90-line event-name mapping:

Job	Claude Code	Cursor	Gemini CLI	OpenCode	Budget
Register; inject peer digest	`SessionStart`	`sessionStart`	`SessionStart`	session event	500ms
Check foreign claims; warn	`PreToolUse`	`preToolUse`	`BeforeTool`	tool.before	100ms
Record claim; update working set	`PostToolUse`	`afterFileEdit`	`AfterTool`	tool.after	100ms
Regenerate digest	`Stop`	`stop`	`AfterAgent`	agent end	300ms
Snapshot before compaction	`PreCompact`	*(none)*	`PreCompress`	*(varies)*	500ms
Release claims; finalize	`SessionEnd`	`sessionEnd`	`SessionEnd`	session end	200ms
Delta injection (V1)	`UserPromptSubmit`	`beforeSubmitPrompt`	`BeforeAgent`	prompt event	200ms

Config locations: `.claude/settings.json`, `.cursor/hooks.json`, `.gemini/settings.json`, OpenCode TypeScript plugin. `init` writes to whichever it detects.

Gemini CLI's hooks are enabled by default from v0.26.0+ and explicitly support context injection (`BeforeAgent`, `BeforeModel`). OpenCode exposes 25+ lifecycle events with tool-execution interception via TypeScript plugins.

Codex CLI is the only major agent with no user-facing hook support — it runs at T1, which is fully functional.

Every hook exits 0 unconditionally. A hook that fails, times out, or finds a corrupt store injects nothing and returns success. Principle 7 is enforced at the process level, not by convention.

10.8 Data model

```
CREATE TABLE sessions (
  session_id   TEXT PRIMARY KEY,
  pid          INTEGER,
  name         TEXT,
  agent        TEXT,          -- 'claude-code' | 'cursor' | 'codex' | ... from clientInfo
  agent_ver    TEXT,
  tier         INTEGER,       -- 1 = MCP only, 3 = hook-enhanced
  cwd          TEXT,
  workspace    TEXT,          -- resolved workspace root
  started_at   INTEGER,
  last_seen    INTEGER,       -- heartbeat
  status       TEXT,          -- active | idle | ended
  intent       TEXT           -- distilled from first user prompt
);

CREATE TABLE claims (
  path        TEXT,
  session_id  TEXT,
  claimed_at  INTEGER,
  expires_at  INTEGER,
  op          TEXT,           -- edit | write | reserve
  PRIMARY KEY (path, session_id)
);

CREATE TABLE digests (
  session_id  TEXT PRIMARY KEY,
  updated_at  INTEGER,
  intent      TEXT,
  files       TEXT,           -- JSON array
  commands    TEXT,           -- JSON array
  last_msg    TEXT,           -- from Stop.last_assistant_message
  tokens      INTEGER         -- measured cost of this digest
);

CREATE TABLE notes (              -- durable findings, survive session death
  id          INTEGER PRIMARY KEY,
  session_id  TEXT,
  workspace   TEXT,
  created_at  INTEGER,
  text        TEXT,
  tags        TEXT
);
```

```
CREATE VIRTUAL TABLE notes_fts USING fts5(text, content=notes);

CREATE TABLE events (             -- local effectiveness telemetry only
  ts          INTEGER,
  session_id  TEXT,
  kind        TEXT,             -- inject | warn | pull | collision | reap
  tokens      INTEGER
);
```

Store lives at `<workspace>/.switchboard/state.db`, added to `.git/info/exclude`.

10.9 Key technical decisions

ADR-0 — MCP is the foundation; hooks are an enhancement. The original design made Claude Code hooks the substrate, which would have required an adapter per agent and excluded every agent without hooks. Inverting it — MCP server as registry, hooks as upgrade — makes cross-agent support *structural* rather than *maintained*. An MCP-speaking agent released next year works with no code change. **This single inversion is what "native across all applications" actually means.**

ADR-0b — Adopt AGENTS.md; do not invent a context-file format. AGENTS.md is stewarded by the Linux Foundation's Agentic AI Foundation, read natively by 30+ agents, present in 60k+ repos. Inventing a `.switchboardrc` would mean begging 30 vendors for support. Instead: one static stanza in a file they already read. The churning state goes in a git-excluded file the stanza points at, so the repo never sees diff noise.

ADR-1 — Push a thin roster, pull rich detail. Pushing everything pollutes context and burns tokens on sessions that don't care. Pulling everything means the agent may never call the tool. **Resolution: push ~200–500 tokens of *pointers* (who's live, what they claim, one finding each) and expose depth via MCP tools.** The pushed block's most valuable content is the instruction to pull. This is the central context-economics decision and everything in §9.2 follows from it.

ADR-2 — Mechanical digests before LLM digests. An LLM summarization step adds latency, cost, nondeterminism, and a failure mode inside a hook with a 300ms budget. Files touched, commands run, and `last_assistant_message` are already structured in the hook payload. **Start mechanical. Add Haiku only if mechanical proves measurably too thin** (§18, M2 gate).

ADR-3 — Observe claims, never ask for them. `PostToolUse` fires on every successful edit regardless of whether the model remembered to announce it. Self-reporting via a tool the agent must choose to call is unreliable by construction.

ADR-4 — Warn, never block. A `PreToolUse` `deny` on a false positive stops real work and produces an uninstall. `additionalContext` costs ~40 tokens and lets the model decide. Blocking is opt-in via `--strict` and off by default.

ADR-5 — SQLite over JSON files. JSON + lockfile means hand-rolling atomicity across concurrent writers. SQLite WAL gives ACID, concurrent reads during a write, and FTS5 for free, via `node:sqlite` with zero dependencies. `better-sqlite3` is the fallback for Node <22 — a compiled dep, which is the one place adding a dependency beats writing code.

ADR-6 — Delegate delivery to native messaging. `sb_handoff` composes a digest and hands it to `SendMessage`. Switchboard owns *what to say*; the platform owns *getting it there*. Inheriting Anthropic's inbound controls, rate limits, permission boundaries, and loop protection for free is worth more than any transport we'd write.

ADR-7 — Workspace identity: `cwd`, then IDE lock, then git root. `cwd` is always present. `~/.claude/ide/*.lock` gives true multi-folder VS Code workspaces. Git root (V2) unifies worktrees. Resolve in that order, cache per session.

ADR-8 — Hooks fail open, always. Every hook is wrapped so any throw, timeout, or corrupt read exits 0 with empty stdout. A tool that shares context must never be why a session won't start.

10.10 Context economics

This is the discipline that decides whether the product works.

The cost side. The injected digest is pure overhead on every session start. At 500 tokens × 8 sessions/day × 30 days ≈ 120k tokens/month of pure injection.

The benefit side (hypothesis, to be validated — not a measured claim). A fresh session reaching working understanding of a subsystem typically reads 5–10 files and runs several searches: on the order of 20–50k tokens of rediscovery. If a 400-token digest eliminates even *half* of one such cycle, it returns roughly 25× its cost.

The falsifiable bet: *injected tokens < tokens saved through avoided rediscovery.* If M2 (§18) shows otherwise, the digest shrinks or the product is wrong. This number gets instrumented in the `events` table from day one, because a product whose core claim is unmeasured is a product that can't be corrected.

Hard caps, enforced in code:

- `SessionStart` digest: 500 tokens, absolute
- Per-peer entry: 80 tokens
- `PreToolUse` warning: 60 tokens
- `UserPromptSubmit` delta: 150 tokens, max once per 5 turns
- No live peers → **zero tokens**

---

11. Cross-application compatibility

11.1 The support matrix

Everything below is **V0 scope**. There is no per-agent adapter and no phased agent rollout — support follows from MCP and AGENTS.md, both of which these agents already implement.

Agent	MCP	Hooks	AGENTS.md	Tier	Discovery	Roster	Pull tools	Auto-inject	Observed claims
Claude Code	✅	✅	via `@import`	**T3**	✅	✅	✅	✅	✅
Cursor	✅	✅ 1.7+	✅	**T3**	✅	✅	✅	✅	✅
Gemini CLI	✅	✅ v0.26+	✅	**T3**	✅	✅	✅	✅	✅
OpenCode	✅	✅ plugins	✅	**T3**	✅	✅	✅	✅	✅
Windsurf	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Codex CLI	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
GitHub Copilot / CLI	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Cline / Roo / Kilo	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Zed	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Continue.dev	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Goose / Qwen / Amazon Q	✅	❌	✅	T1	✅	✅	✅	via T0/T2	voluntary
Aider / Zed / Jules / Devin	varies	❌	✅	T0–T1	if MCP	✅	if MCP	via T0/T2	voluntary
Any future MCP agent	✅	—	—	T1	✅	✅	✅	via T0/T2	voluntary

Read the last row as the point of the design. An agent that ships next quarter and speaks MCP is supported on day one, with no release from us.

11.2 Mixed-agent workspaces are a first-class case

The roster is agent-labelled, because heterogeneous workspaces are normal — a Claude Code session for refactoring, a Cursor window for UI, a Codex CLI run for a migration:

```
[switchboard] 3 live sessions in this workspace:

• auth-fix        [claude-code]  active 40m — debugging login 500s
  editing: middleware/session.ts, lib/jwt.ts

• ui-polish       [cursor]       active 8m  — checkout form styling
  editing: components/Checkout.tsx

• data-migration  [codex]        active 22m — backfilling tenant_id
  claimed: scripts/backfill.ts  (self-declared)
```

`(self-declared)` marks a T1 voluntary claim versus an observed T3 one. The distinction is surfaced, not hidden, so the reader knows how much to trust it.

11.3 Config fragmentation is the real portability tax

MCP is universal; **MCP *configuration* is not.** Every agent uses a different file and format:

Agent	MCP config location
Claude Code	`.mcp.json` / `.claude/settings.json`
Cursor	`.cursor/mcp.json`
Windsurf	`~/.codeium/windsurf/mcp_config.json`
Codex CLI	`~/.codex/config.toml`
Gemini CLI	`.gemini/settings.json`
VS Code / Copilot	`.vscode/mcp.json`
Cline / Roo	extension storage JSON

`init` detects installed agents by probing these paths and writes the server entry into each one found, reporting exactly what it touched. This is the one genuinely tedious part of the implementation — **~200 lines of writer functions, no cleverness available.** It is also why `init` must be idempotent, diff-printing, and fully reversible.

11.4 Version floors

Capability	Requirement
Core (T1)	any MCP client, stdio transport
Auto-inject, observed claims	Claude Code v2.1.0+ · Cursor v1.7+ · Gemini CLI v0.26+ · OpenCode (plugin API)
`sb_handoff` over native messaging	Claude Code v2.1.224+
`PreCompact` rescue	Claude Code only

`doctor` reports the tier reached per detected agent. Features degrade individually; nothing fails wholesale.

11.5 Agent Teams coexistence

If `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`, Switchboard detects teammates via the team config `members` array and marks them `team:<lead>` — visible, not duplicated. The features are complementary: Agent Teams coordinates *planned* work within one agent; Switchboard covers *unplanned* work across all of them.

11.6 Worktrees

Sessions in sibling worktrees share a git root but differ by `cwd`. V0 treats them as separate workspaces (correct and safe). V1 adds `--link-worktrees`.

11.7 Subagents

Subagent hook payloads carry `agent_id` / `agent_type`. Subagent activity attributes to the parent session and never appears as a separate roster row — five subagents must not become five roster entries.

11.8 Sandboxing

Sandboxed shells may not reach the store or socket. `doctor` detects and reports this explicitly rather than failing silently.

11.9 Operating systems

macOS and Linux at V0, WSL2 included. Native Windows works at T1 (MCP is cross-platform); only the `sb_handoff` native-messaging path is Windows-gated, since it depends on named pipes and a different home directory.

---

12. Failure modes

Failure	Consequence	Mitigation
Phantom claim from a crashed session	New sessions warned about a file nobody owns	PID liveness + TTL + reap on every hook
Digest is noise, not signal	Agents get worse; trust collapses	Mechanical extraction from structured payloads; token caps; M2 kill gate
Store corruption	Every session start breaks	Fail open (ADR-8); `doctor --repair` recreates the store
Concurrent write contention	Hook timeouts	WAL mode; writes <5ms; `busy_timeout` 50ms then give up silently
Hook API drift	Product silently stops working	Thin adapter layer (§17 RISK-1); `doctor` validates payload shape
Context bloat over many peers	10 peers × 80 tokens blows the cap	Hard cap; recency-order; collapse beyond 5 into a count
Digest leaks a secret between sessions	Credential exposure across contexts	Redact common secret patterns before write (§16)
User uninstalls, hooks remain	Broken session starts	`uninstall` is P0, and `init` records exactly what it added for clean reversal
Two sessions warn each other into paralysis	Deadlock-by-politeness	Warnings are advisory only; never blocking (ADR-4)

13. Security and privacy

Threat model. The store contains distilled transcript content: file paths, commands, findings — plausibly including secrets, customer data, and unreleased code.

Guarantees:

1. **Local only.** No network calls in the data path. Ever. No content telemetry.
2. **No listening socket.** Switchboard binds nothing. `claude-mem`'s unauthenticated port 37777 — rated HIGH risk in a February 2026 community audit — is the exact mistake this architecture makes structurally impossible. There is no server to leave unauthenticated.
3. **Filesystem permissions.** Store is `0600`, owned by the invoking user. Same trust boundary as `~/.claude` itself.
4. **Workspace-scoped.** A session in project X never sees project Y's state. Enforced at query level, not display level.
5. **Secret redaction on write.** Pattern-scan digests before persisting: `sk-`, `ghp_`, AWS key shapes, `Bearer`, `PRIVATE KEY`, high-entropy strings in assignment position. Imperfect, and documented as imperfect.

6. `.git/info/exclude`**, never** `.gitignore`**.** The store must never be committed, and the user's tracked files must never be modified by an installer.
7. **Inspectable and deletable.** `switchboard status --raw` dumps everything stored. `switchboard purge` deletes it. No hidden state.
8. **Prompt-injection boundary.** Digest content originates from other sessions' output and is therefore *untrusted input*. It is injected wrapped in an explicit envelope marking it as peer-reported data, never as instructions — mirroring how Claude Code itself frames incoming cross-session messages.

---

14. Metrics

North star: time-and-tokens to productive first edit in a fresh session. From session open to first correct edit. Measured with Switchboard on vs. off. This is the number the whole product lives or dies by.

Primary:

Metric	Definition	Target
Rediscovery reduction	Tokens spent on file reads/greps in the first 5 turns of a fresh session	–30%
Human relay events	User prompts containing >200 chars of pasted paths/code/output	–50%
Collision rate	Distinct sessions editing the same file within 10 min	–60%
Pull-through rate	% of primed sessions that call an `sb_*` tool within 10 turns	>25%

Pull-through is the sharpest signal: it means the digest was *interesting enough to follow up on*. A high injection rate with near-zero pull-through means we're writing tokens nobody reads.

Guardrails (regressions that halt the roadmap):

Guardrail	Threshold
Injected tokens / saved tokens	must stay <0.2
`SessionStart` p95 latency added	<300ms
False-positive collision warnings	<10% of warnings
Sessions where injection is empty (no peers)	must be 100% zero-token

Qualitative kill signal: if users report agents behaving *worse* after install, stop and shrink the digest. Context pollution is the #1 way this product fails, and it will not show up in aggregate token counts.

---

15. Rollout

Milestone	Scope	Gate to proceed
M0 — Spike (3 days)	Prove `SessionStart` injection + peer discovery end-to-end in one workspace	A second session visibly knows about the first
M1 — V0 dogfood (2 wks)	Full V0 on your own `logline` workspace — 20 sessions, real conditions	1 week of daily use, zero session-start failures
M2 — Effectiveness gate (1 wk)	Instrument the token hypothesis (§10.7) with real data	**Injected < saved. If false, shrink or stop.**
M3 — Private alpha (2 wks)	10–20 heavy multi-session users	≥25% pull-through; no "agent got worse" reports
M4 — Public	npm, docs, `doctor`, uninstall	—
M5 — V1	Handoff, notes, PreCompact rescue, board TUI	—

M2 is a genuine kill gate, not a checkpoint. If a 400-token digest does not measurably reduce rediscovery, the honest outcome is to ship only the collision guard — which is independently valuable, costs ~60 tokens, and needs no digest at all.

---

16. Risks

ID	Risk	Severity	Mitigation
RISK-1	`~/.claude/sessions/*.json` and `ide/*.lock` are **undocumented internals** and can change without notice	**High**	Isolate every read behind one `platform/` adapter (~100 LOC). Version-gate. `doctor` validates shape on every run and warns loudly on drift. Fall back to `ps`-based discovery. **Accept this risk knowingly — it is the price of not building a daemon, and it is the right trade.**
RISK-2	Anthropic ships ambient shared state natively	High	Likely, eventually — they shipped the transport in one release. Mitigations: move fast, stay small, be the thing that's easy to delete. A tool that made itself obsolete by proving demand is a successful experiment, not a failure.
RISK-3	Context pollution makes agents worse	**High**	Hard caps; pointers-not-payloads (ADR-1); M2 gate; qualitative kill signal
RISK-4	Nobody installs it — friction at step two	Medium	Single `npx` command, no accounts, no config, `--dry-run`, clean uninstall
RISK-5	Digests too thin to be useful	Medium	ADR-2 escalation path to Haiku summarization, gated on M2 data
RISK-6	Hook overhead degrades session start	Medium	Strict budgets; yield-and-inject-nothing on timeout; p95 guardrail
RISK-7	Secret leaks between session contexts	Medium	Redaction on write; local-only; documented as best-effort
RISK-8	Agent Teams double-counts sessions in roster	Low	Detect team config; tag rather than duplicate (§11.5)
RISK-9	**MCP config fragmentation** — 7+ agents, 7 different file formats and locations, each free to change	**Medium**	One writer function per agent (~200 LOC total), isolated in `installers/`. `init` is idempotent and diff-printing; `doctor` re-validates. Failure is per-agent and loud, never silent or global. This is the honest cost of cross-agent support.
RISK-10	T1 voluntary claims are unreliable — model doesn't call `sb_claim`	Medium	Roster marks `(self-declared)` vs observed. T0 stanza instructs explicitly. Accepted tier difference (§10.4), never oversold.
RISK-11	AGENTS.md stanza edits a tracked file	Low	Written **once** at `init`, static, ~5 lines. All churning state lives in git-excluded `.switchboard/ROSTER.md`. `uninstall` removes the stanza.

17. Open questions

1. **Name.** "Switchboard" is likely taken on npm. Alternatives: `commons`, `partyline`, `hivemind`, `crosstalk`. Needs a name-and-scope decision before M4.
2. **Does the digest need the *why*?** Files and commands are mechanical. Intent ("we're migrating off Redis") may require either an LLM pass or explicit `sb_note` calls. M2 should answer this.
3. **Claim TTL.** 15 minutes is a guess. Too short → phantom-free but useless. Too long → false positives. Instrument and tune.
4. **Should idle sessions appear in the roster?** A session idle 4 hours is arguably dead. Proposal: show, but de-emphasize and truncate first.
5. **Opt-in vs. opt-out per session?** Some sessions are private (credentials work, spikes). Proposal: `--no-switchboard` flag; possibly auto-exclude sessions in `bypassPermissions` mode.
6. **Is the collision guard the real product?** It's cheaper, more obviously valuable, and needs no digest infrastructure. If M2 fails, this is the fallback product — and it might be the better one regardless.
7. **License and business model.** MIT and free is the fastest path to adoption and the honest answer for a tool this thin. Anything else needs a reason.

---

18. Appendix A — Verified platform facts

Confirmed 2026-08-25 against live documentation and this machine.

Hooks with `hookSpecificOutput.additionalContext`: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PostToolBatch`, `Stop`, `SubagentStop`.

Common hook payload fields: `session_id`, `transcript_path`, `cwd`, `hook_event_name`, `permission_mode`. Tool hooks add `tool_name`, `tool_input`, `tool_use_id`, `tool_result`, and optional `agent_id` / `agent_type`.

`SessionStart` **matchers:** `startup`, `resume`, `clear`, `compact`, `fork`.

`Stop` **payload** includes `last_assistant_message` — the primary mechanical digest source (FR-6.2).

`SessionEnd` shares a ~1.5s budget across all registered SessionEnd hooks.

`PreToolUse` supports `permissionDecision: allow|deny`; timeout does not block.

Env vars exported to hooks: `CLAUDE_CODE_MESSAGING_SOCKET`, `CLAUDE_CODE_MESSAGING_TOKEN` — present when cross-session messaging is enabled; exported before any hook runs, including `SessionStart`.

Session registry: `~/.claude/sessions/<pid>.json` → `pid`, `sessionId`, `cwd`, `startedAt`, `version`, `kind`, `name`, `status`, `updatedAt`, `waitingFor`.

IDE binding: `~/.claude/ide/<pid>.lock` → `pid`, `workspaceFolders[]`, `ideName`, `transport`, `authToken`.

Transcripts: `~/.claude/projects/<slugified-cwd>/<session-id>.jsonl`, appended live. Observed 3–14 MB per session — **too large to read in a hook.** Use hook payloads, not transcript parsing. (This is why FR-6.2 extracts from payloads.)

This machine: Claude Code v2.1.223 — **below the v2.1.224 floor for cross-session messaging.** Upgrade before validating §4.

Cross-agent facts

AGENTS.md — released by OpenAI Aug 2025; transferred to the Linux Foundation's **Agentic AI Foundation** Dec 9, 2025. By May 2026: **170+ member organisations, 60,000+ repositories, 30+ agents reading it natively** — Codex, Cursor, Copilot, Windsurf, Gemini CLI, Aider, Zed, VS Code, Jules, Devin, Factory, Amp, Junie. **Claude Code is the exception**, still loading `CLAUDE.md`; Anthropic's own docs recommend bridging via a first-line `@AGENTS.md` import or `ln -s AGENTS.md CLAUDE.md`.

MCP client support — Claude Code, Cursor, Windsurf, Cline, OpenCode, Continue.dev, Roo Code, Kilo Code, Void AI, Codex CLI, Gemini CLI, GitHub Copilot CLI, Goose, Qwen Code, Amazon Q Developer CLI (Kiro), Antigravity CLI, Grok Build.

Cursor hooks (v1.7+, Oct 2025) — `sessionStart`, `sessionEnd`, `preToolUse`, `postToolUse`, `postToolUseFailure`, `subagentStart`, `subagentStop`, `beforeShellExecution`, `afterShellExecution`, `beforeMCPExecution`, `afterMCPExecution`, `beforeReadFile`, `afterFileEdit`, `beforeSubmitPrompt`, `stop`. Config: `$REPO/.cursor/hooks.json` or `~/.cursor/hooks.json`. **Near-identical shape to Claude Code's** — one implementation covers both.

Codex CLI — as of mid-2026, the only major platform with **no user-facing hook support**. Runs at T1.

MCP stdio lifecycle — client spawns one server process per session and terminates it at session end; `initialize` carries `clientInfo: {name, version}`. This is the basis of ADR-0.

---

19. Appendix B — Sources

- Cross-session messaging — Claude Code Docs
- Agent teams — Claude Code Docs
- Hooks reference — Claude Code Docs
- Issue #21277 — Cross-context communication between Claude Code instances
- Claude Code sessions can now talk to each other — the-decoder
- session-bridge — Shreyas Patil
- Claude-Mem guide — DataCamp
- Mem0 Claude Code integration
- Inside Claude Code's shared task list — MindStudio
- Session start hooks — MindStudio
- Hooks — Cursor Docs
- Cursor 1.7 adds hooks for agent lifecycle control — InfoQ

- The AGENTS.md Field Guide, 2026 edition
- AGENTS.md vs CLAUDE.md vs Cursor Rules vs Copilot (2026)
- Agent instruction files and cross-tool portability — Codex KB
- AI Harness Engineering Compatibility Matrix — June 2026
- CLI coding agents comparison — hidekazu-konishi.com
- Best MCP clients in 2026 — Nimbalyst